

# LAB SHEET

## STOCK ANALYSIS USING MAPREDUCE

*Hadoop 3.3.6 • WSL Ubuntu • Java MapReduce*

### 1. Objective

To implement a Stock Analysis MapReduce program that reads stock-market CSV data from HDFS, extracts the stock symbol and closing price in the Mapper, and calculates the average closing price for each stock in the Reducer.

### 2. Software / Environment

- WSL Ubuntu
- Apache Hadoop 3.3.6
- Java
- Eclipse (optional; the program can also be created directly from the terminal)
- HDFS and YARN

### 3. Lab Directory Setup

#### 1. Open the Hadoop workspace

```
cd ~/eclipse-workspace
hadoop@Suhel-PC:~$ cd ~/eclipse-workspace/
hadoop@Suhel-PC:~/eclipse-workspace$ ls
WordCCountFinal.txt WordCount.jar myfile sc1 wc1
```

#### 2. Create the StockAnalysis project directory

```
mkdir StockAnalysis
```

#### 3. Enter the project directory

```
cd StockAnalysis
hadoop@Suhel-PC:~/eclipse-workspace$ mkdir StockAnalysis
hadoop@Suhel-PC:~/eclipse-workspace$ cd StockAnalysis/
```

#### 4. Set Hadoop classpath

```
export HADOOP_CLASSPATH=$(hadoop classpath)
hadoop@Suhel-PC:~/eclipse-workspace/StockAnalysis$ export HADOOP_CLASSPATH=$(hadoop classpath)
```

### 4. Create StockMapper.java

Create the Mapper source file:

```
nano StockMapper.java
```

```
hadoop@Suhe1-PC:~/eclipse-workspace/StockAnalysis$ cat StockMapper.java
```

Enter the following Java code:

```
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

public class StockMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {

    private Text stockSymbol = new Text();
    private DoubleWritable closePrice = new DoubleWritable();

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        // Skip header
        if (line.startsWith("Date")) {
            return;
        }

        String[] fields = line.split(",");

        String stock = fields[1];
        double close = Double.parseDouble(fields[5]);

        stockSymbol.set(stock);
        closePrice.set(close);

        context.write(stockSymbol, closePrice);
    }
}
```

Save in nano: Ctrl+O → Enter, then Ctrl+X.

## 5. Create StockReducer.java

```
nano StockReducer.java
```

```
hadoop@Suhe1-PC:~/eclipse-workspace/StockAnalysis$ nano StockReducer.java
```

Enter the following Java code:

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;

public class StockReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    @Override
    protected void reduce(Text key, Iterable<DoubleWritable> values, Context context)
        throws IOException, InterruptedException {

        double sum = 0.0;
        int count = 0;
```

```

        for (DoubleWritable value : values) {
            sum += value.get();
            count++;
        }

        double average = sum / count;

        context.write(key, new DoubleWritable(average));
    }
}

```

Save in nano: Ctrl+O → Enter, then Ctrl+X.

## 6. Create StockAnalysis.java (Driver)

```
nano StockAnalysis.java
```

```
hadoop@Suhel-PC:~/eclipse-workspace/StockAnalysis$ nano StockAnalysis.java
```

Enter the following Java code:

```

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class StockAnalysis {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.err.println("Usage: StockAnalysis <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Stock Analysis");

        job.setJarByClass(StockAnalysis.class);

        job.setMapperClass(StockMapper.class);
        job.setReducerClass(StockReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Save in nano: Ctrl+O → Enter, then Ctrl+X.

## 7. Check Source Files

```
ls
```

Expected files:

```
StockAnalysis.java
StockMapper.java
StockReducer.java
```

## 8. Compile the Java Program

### 5. Set the Hadoop classpath

```
export HADOOP_CLASSPATH=$(hadoop classpath)
```

```
hadoop@Suhel-PC:~/eclipse-workspace/StockAnalysis$ echo $HADOOP_CLASSPATH
/home/hadoop/hadoop/etc/hadoop:/home/hadoop/hadoop/share/hadoop/common/lib/*
:/home/hadoop/hadoop/share/hadoop/common/*:/home/hadoop/hadoop/share/hadoop/
hdfs:/home/hadoop/hadoop/share/hadoop/hdfs/lib/*:/home/hadoop/hadoop/share/h
adoop/hdfs/*:/home/hadoop/hadoop/share/hadoop/mapreduce/*:/home/hadoop/hadoo
p/share/hadoop/yarn:/home/hadoop/hadoop/share/hadoop/yarn/lib/*:/home/hadoop
/hadoop/share/hadoop/yarn/*:/home/hadoop/hadoop/share/hadoop/common/lib/anim
al-sniffer-annotations-1.17.jar:/home/hadoop/hadoop/share/hadoop/common/lib/
audience-annotations-0.5.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/a
vro-1.7.7.jar:/home/hadoop/hadoop/share/hadoop/common/lib/checker-qual-2.5.2
.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-beanutils-1.9.4.jar
:/home/hadoop/hadoop/share/hadoop/common/lib/commons-cli-1.2.jar:/home/hadoo
p/hadoop/share/hadoop/common/lib/commons-codec-1.15.jar:/home/hadoop/hadoop/
share/hadoop/common/lib/commons-collections-3.2.2.jar:/home/hadoop/hadoop/sh
are/hadoop/common/lib/commons-compress-1.21.jar:/home/hadoop/hadoop/share/ha
adoop/common/lib/commons-configuration2-2.8.0.jar:/home/hadoop/hadoop/share/h
adoop/common/lib/commons-daemon-1.0.13.jar:/home/hadoop/hadoop/share/hadoop/
common/lib/commons-io-2.8.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/
commons-lang3-3.12.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons
-logging-1.1.3.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-math3
-3.1.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-net-3.9.0.jar
:/home/hadoop/hadoop/share/hadoop/common/lib/commons-text-1.10.0.jar:/home/h
adoop/hadoop/share/hadoop/common/lib/curator-client-5.2.0.jar:/home/hadoop/h
adoop/share/hadoop/common/lib/curator-framework-5.2.0.jar:/home/hadoop/hadoo
p/share/hadoop/common/lib/curator-recipes-5.2.0.jar:/home/hadoop/hadoop/shar
e/hadoop/common/lib/dnsjava-2.1.7.jar:/home/hadoop/hadoop/share/hadoop/commo
n/lib/failureaccess-1.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/gson
-2.9.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/guava-27.0-jre.jar:/h
```

### 6. Compile all three Java files

```
javac -classpath "$HADOOP_CLASSPATH" -d . StockMapper.java StockReducer.java
StockAnalysis.java
```

### 7. Check generated class files

```
find . -name "*.class"
```

```
hadoop@Suhel-PC:~/eclipse-workspace/StockAnalysis$ find . -name "*.class"
./StockMapper.class
./StockAnalysis.class
./StockReducer.class
```

## 9. Create the MapReduce JAR

### 8. Create StockAnalysis.jar

```
jar -cvf StockAnalysis.jar StockMapper.class StockReducer.class StockAnalysis.class
hadoop@Suhel-PC:~/eclipse-workspace/StockAnalysis$ jar -cvf StockAnalysis.jar StockMapper.class StockReducer.class StockAnalysis.class
added manifest
adding: StockMapper.class(in = 1942) (out= 805)(deflated 58%)
adding: StockReducer.class(in = 1639) (out= 693)(deflated 57%)
adding: StockAnalysis.class(in = 1538) (out= 854)(deflated 44%)
```

### 9. Check the JAR

```
ls -lh StockAnalysis.jar
hadoop@Suhel-PC:~/eclipse-workspace/StockAnalysis$ ls -lh StockAnalysis.jar
-rw-rw-r-- 1 hadoop hadoop 3.0K Sep 24 04:05 StockAnalysis.jar
```

### 10. Inspect the contents

```
jar tf StockAnalysis.jar
```

Expected class files inside the JAR:

```
StockMapper.class
StockReducer.class
StockAnalysis.class
```

## 10. Create the CSV Dataset

Create the stock dataset:

```
nano stocks.csv
```

Enter exactly the following CSV data:

```
Date,Stock,Open,High,Low,Close,Volume
2024-01-01,GOOG,1450.00,1465.00,1445.00,1460.00,1200000
2024-01-01,AAPL,300.00,305.00,295.00,304.00,2100000
2024-01-02,GOOG,1460.00,1480.00,1450.00,1470.00,1100000
```

Save in nano: Ctrl+O → Enter, then Ctrl+X.

### 11. Verify the CSV

```
cat stocks.csv
```

## 11. Start Hadoop Services

If HDFS is not running, start the Hadoop services:

```
start-dfs.sh
start-yarn.sh
```

You can optionally check running Hadoop processes:

```
jps
```

## 12. Create the HDFS Input Directory

```
hdfs dfs -mkdir -p /input
```

If you receive a connection-refused error, start HDFS first using start-dfs.sh and then repeat the command.

## 13. Upload CSV to HDFS

### 12. Upload the CSV file

```
hdfs dfs -put stocks.csv /input/
```

### 13. Check the HDFS input directory

```
hdfs dfs -ls /input
hadoop@Suhel-PC:~/eclipse-workspace/StockAnalysis$ hdfs dfs -ls /input
Found 1 items
-rw-r--r--  1 hadoop supergroup      202 2026-09-24 03:58 /input/stocks.c
sv
```

### 14. Display the uploaded CSV

```
hdfs dfs -cat /input/stocks.csv
```

## 14. Run the Stock Analysis MapReduce Program

Run the JAR with the HDFS input path and HDFS output path:

```
hadoop jar StockAnalysis.jar StockAnalysis /input/stocks.csv /output/stock_analysis
```

The job should start a MapReduce job. A successful run normally ends with the job state reported as successful. The warning about implementing Tool/ToolRunner can be ignored for this lab; it does not prevent this program from running.

## 15. Check the Output Directory

```
hdfs dfs -ls /output/stock_analysis
```

## 16. Display the MapReduce Result

```
hdfs dfs -cat /output/stock_analysis/part-r-00000
```

For this dataset, the expected result is:

```
AAPL      304.0
GOOG      1465.0
```

Explanation: AAPL has one closing-price value (304.00), so its average is 304.00. GOOG has two closing prices (1460.00 and 1470.00), so its average is  $(1460 + 1470) / 2 = 1465.00$ .

## Optional

### 17. If the Output Directory Already Exists

Hadoop does not normally allow a MapReduce job to write into an existing output directory. Delete the previous output directory and run the job again:

```
hdfs dfs -rm -r /output/stock_analysis
hadoop jar StockAnalysis.jar StockAnalysis /input/stocks.csv /output/stock_analysis
```

### 18. Complete Command Sequence

The following is the compact command sequence for repeating the complete experiment after the Java files and CSV have been created:

```
cd ~/eclipse-workspace/StockAnalysis

export HADOOP_CLASSPATH=$(hadoop classpath)

javac -classpath "$HADOOP_CLASSPATH" -d . StockMapper.java StockReducer.java
StockAnalysis.java

rm -f StockAnalysis.jar

jar -cvf StockAnalysis.jar StockMapper.class StockReducer.class StockAnalysis.class

start-dfs.sh
start-yarn.sh

hdfs dfs -mkdir -p /input
hdfs dfs -put -f stocks.csv /input/

hdfs dfs -ls /input
hdfs dfs -cat /input/stocks.csv

hdfs dfs -rm -r /output/stock_analysis

hadoop jar StockAnalysis.jar StockAnalysis /input/stocks.csv /output/stock_analysis

hdfs dfs -ls /output/stock_analysis
hdfs dfs -cat /output/stock_analysis/part-r-00000
```

### 19. Troubleshooting

#### A. Connection refused on localhost:9000

```
start-dfs.sh
hdfs dfs -mkdir -p /input
```

#### B. Output directory already exists

```
hdfs dfs -rm -r /output/stock_analysis
```

Then rerun the MapReduce command.

### C. Mapper/Reducer type mismatch

Make sure StockMapper uses DoubleWritable as its output value type and StockReducer receives DoubleWritable values:

```
StockMapper extends Mapper<LongWritable, Text, Text, DoubleWritable>  
StockReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable>
```

### D. Check JAR contents

```
jar tf StockAnalysis.jar
```

## 20. Lab Result

The experiment implements a Hadoop MapReduce workflow for stock analysis. The Mapper extracts each stock symbol and closing price, while the Reducer computes the average closing price for each stock. The supplied dataset produces AAPL = 304.0 and GOOG = 1465.0.